Camunda Timer Event

In this session we will look at what is timer event and how it functions ?.

Timer events are events which are triggered by a defined timer. They can be used as start event, intermediate event or boundary event. Boundary events can be interrupting or non interrupting event.

1. Start timer event

A timer start event is used to create process instance at a given time. It can be used both for processes which should start only once and for processes that should start in specific time intervals.

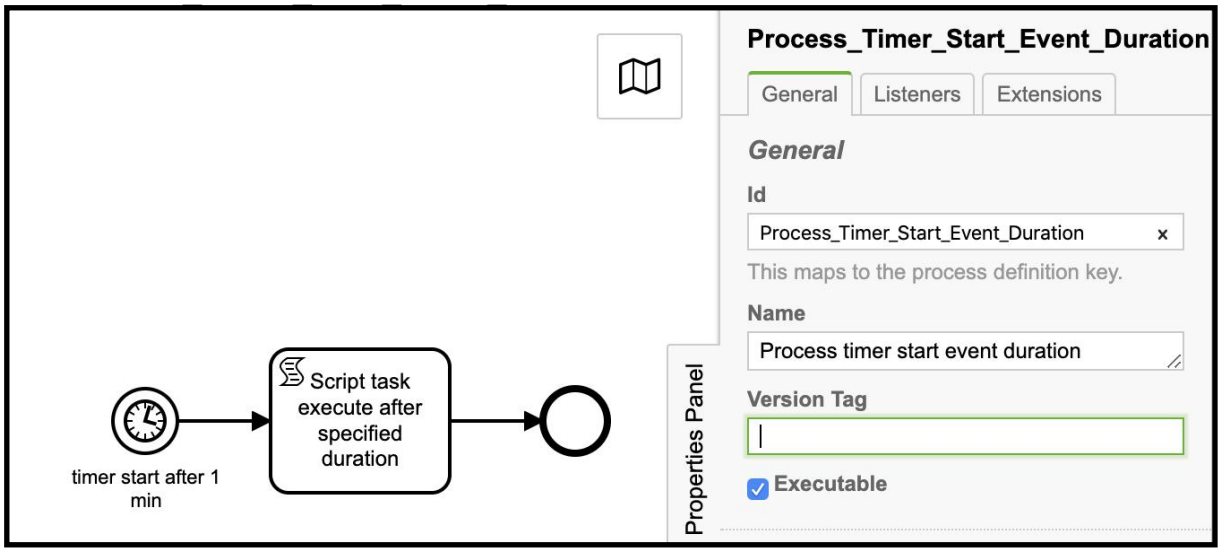
- A sub process cannot have a timer start event.
- A timer start event is scheduled as soon as process is deployed.

Timer definition type: duration

To specify how long the timer should run before it is fired

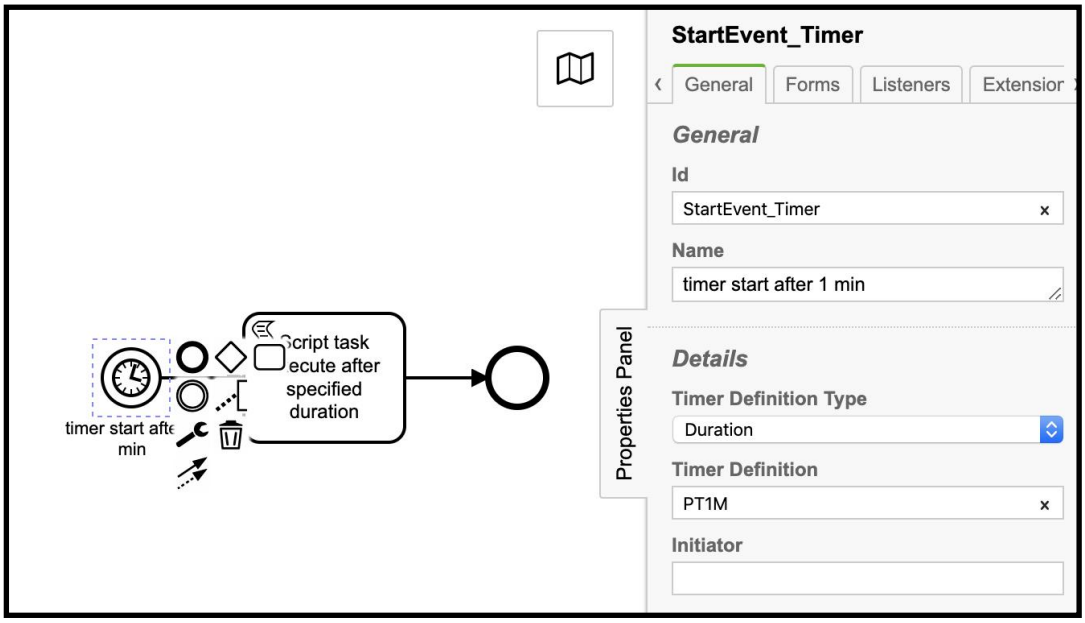
1.1 Timer definition type: Duration

Id: Process_Timer_Start_Event_Duration

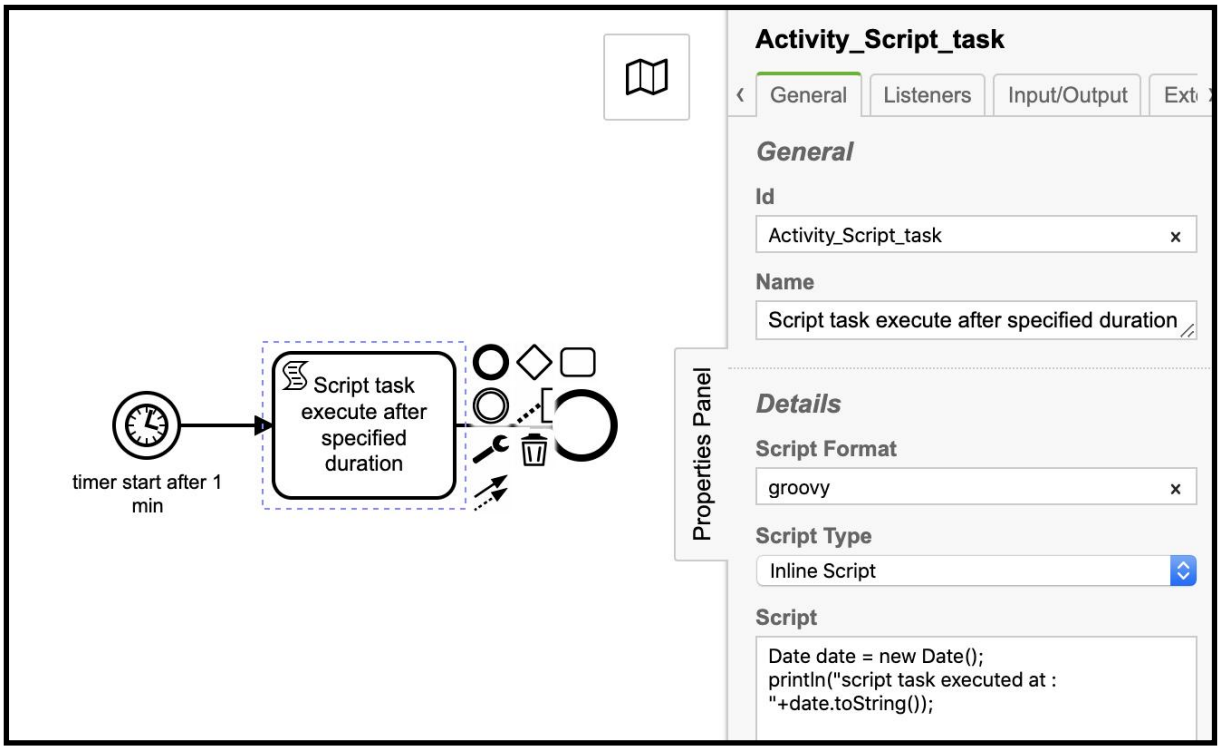


Name: Process timer start event duration

Start timer event



Id: StartEvent_Timer
Name: timer start after 1 min
Timer Definition Type: Duration
Timer Definition: PT1M
Script task: Script task execute after specified duration



Id: Activity_Script_task

Name: Script task execute after specified duration

Script Format: groovy

Script Type: Inline Script

Script:

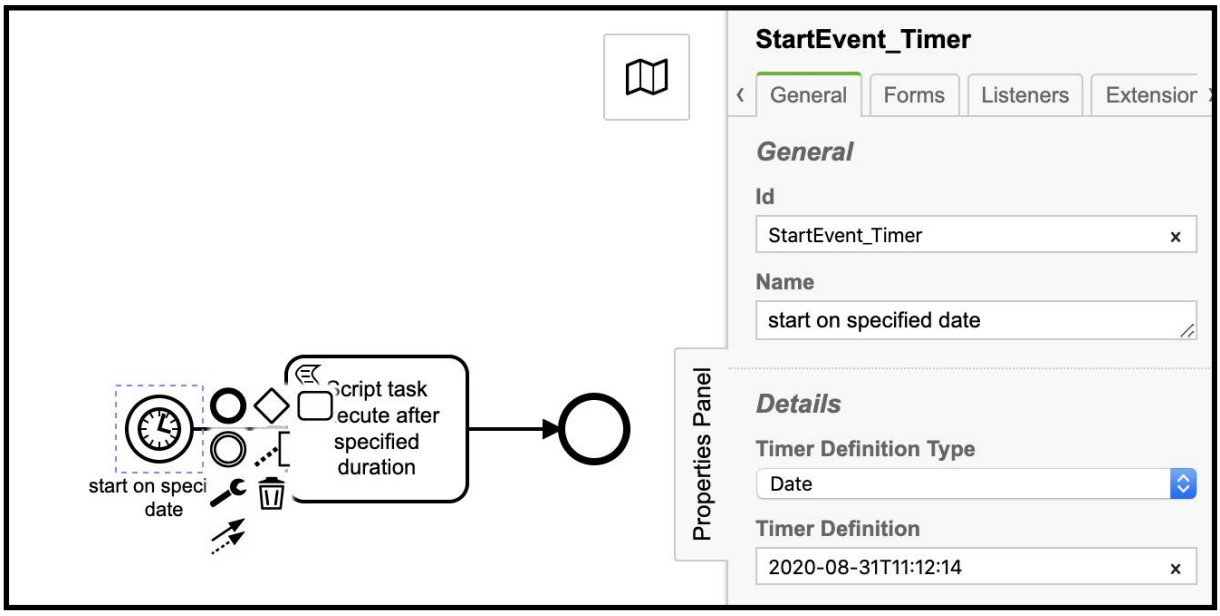
```
Date date = new Date();

println("script task executed at : "+date.toString());
```

Deploy and start the process and observe the console output.

script task executed at : Mon Aug 31 10:43:38 PDT 2020

1.2 Timer definition type: Date



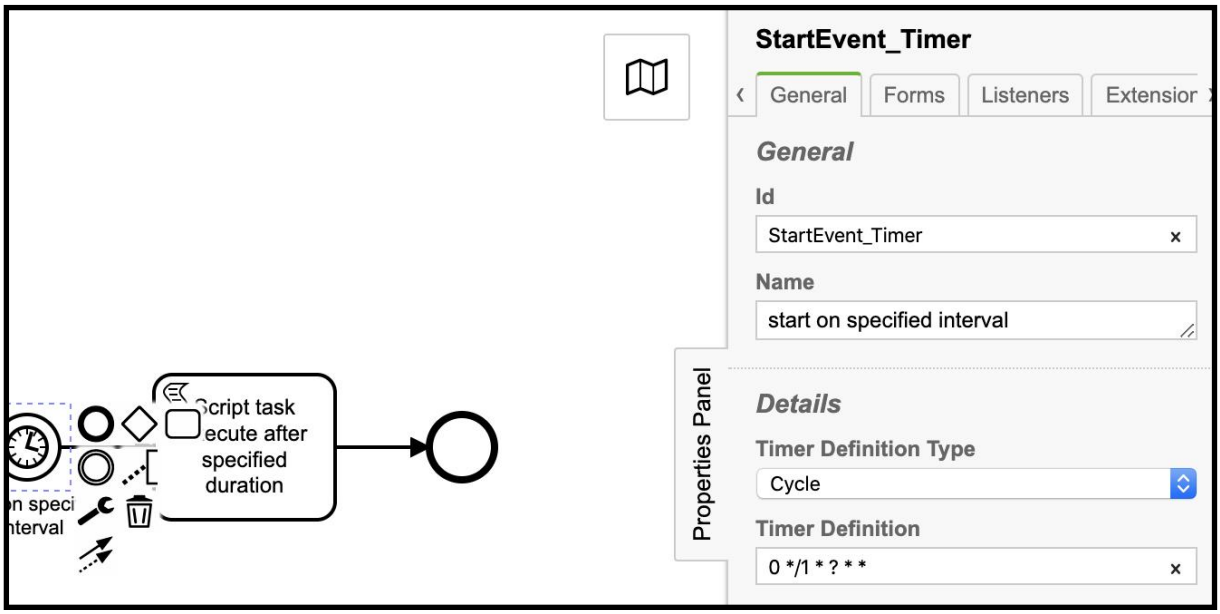
Id: StartEvent_Timer
Name: start on specified date
Timer Definition Type: Date

Timer Definition: 2020-08-31T11:12:14

Deploy and start the the process and observer tomcat console

script task executed at : Mon Aug 31 11:12:40 PDT 2020

1.3 Timer definition type: Cycle



Id: StartEvent_Timer
Name: start on specified interval
Timer Definition Type: Cycle
Timer Definition: 0 */1 * ? * *

As per timer definition crone expression start event will fire on every minute.

Deploy the process and observe console output

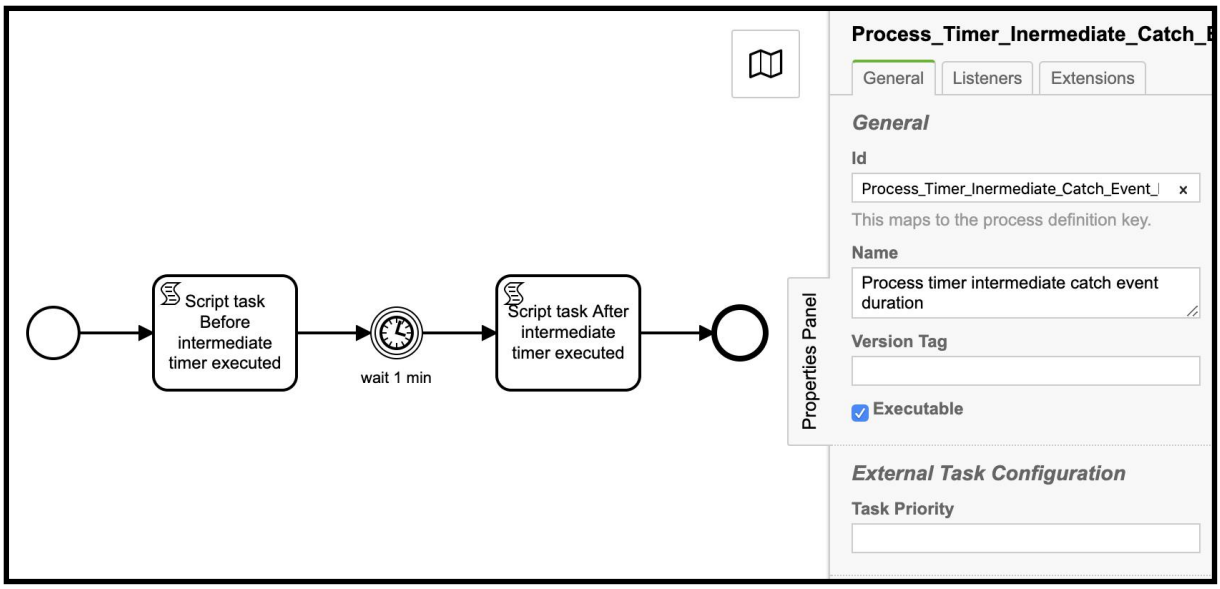
script task executed at : Mon Aug 31 11:44:29 PDT 2020
script task executed at : Mon Aug 31 11:45:05 PDT 2020
script task executed at : Mon Aug 31 11:46:20 PDT 2020
script task executed at : Mon Aug 31 11:47:35 PDT 2020
script task executed at : Mon Aug 31 11:48:10 PDT 2020
script task executed at : Mon Aug 31 11:49:25 PDT 2020

script task executed at : Mon Aug 31 11:50:00 PDT 2020

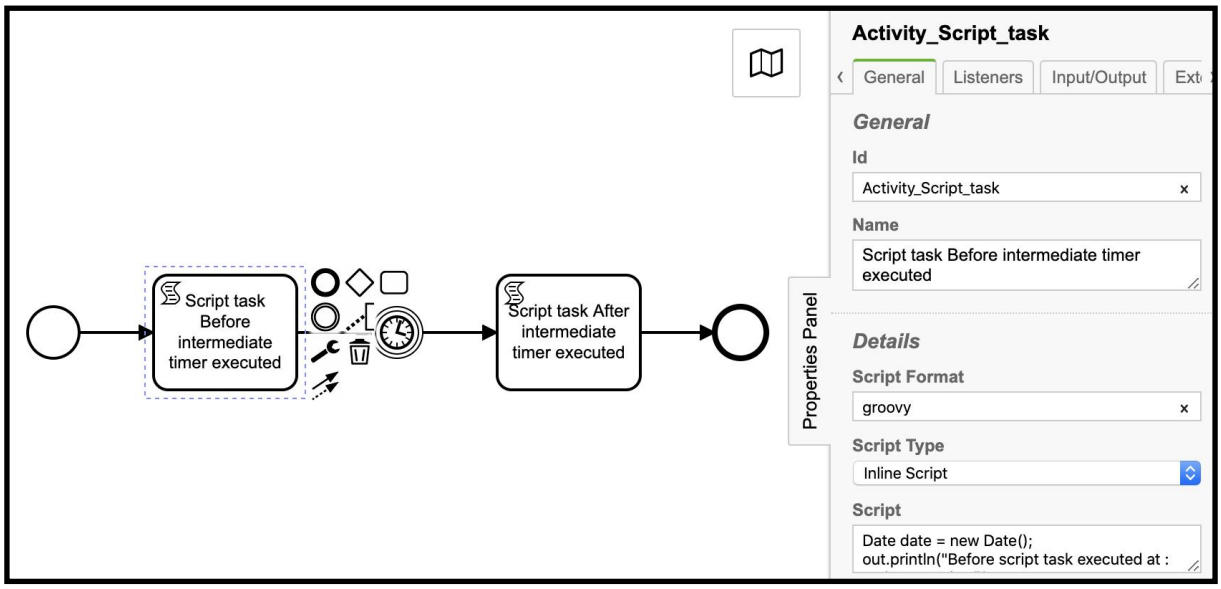
2. Intermediate timer catching event

A timer intermediate event acts as a stopwatch. When an execution arrives in catching event activity, a timer is started. When the timer fires (e.g., after a specified interval), the sequence flow going out of the timer intermediate event is followed.

Id: Process_Timer_Inermediate_Catch_Event_Duration
Name: Process timer intermediate catch event duration



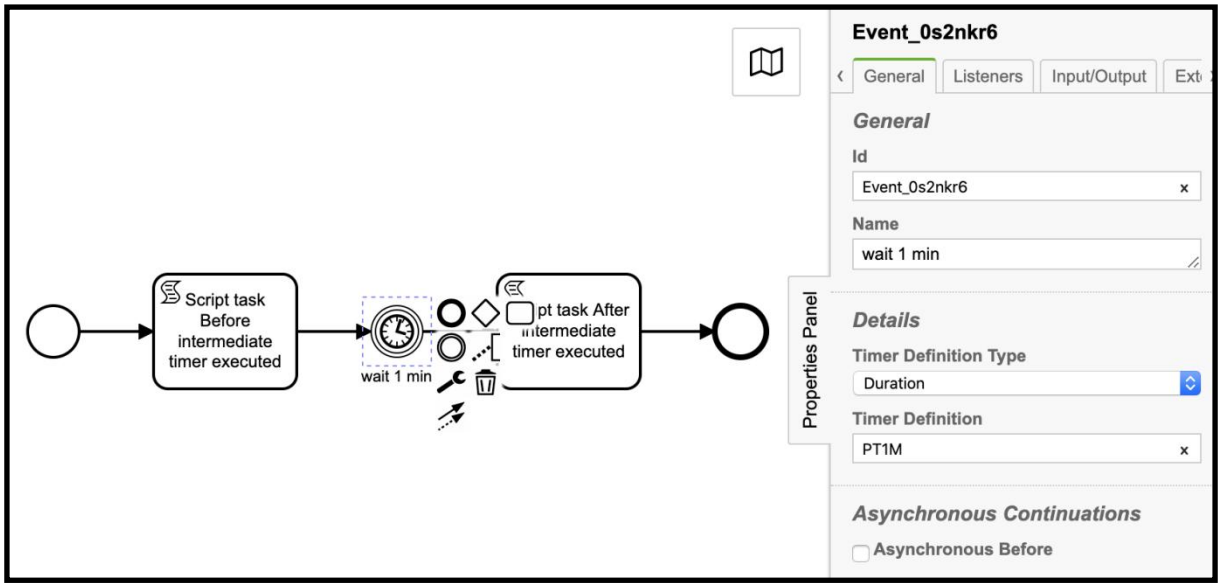
2.1 Script task: Script task Before intermediate timer executed



Id: Activity_Script_task
Name: Script task Before intermediate timer executed
Script Format: groovy
Script Type: Inline Script
Script:

```
Date date = new Date();
out.println("Before script task executed at : “+date.toString());
```

2.2 Timer Intermediate Event



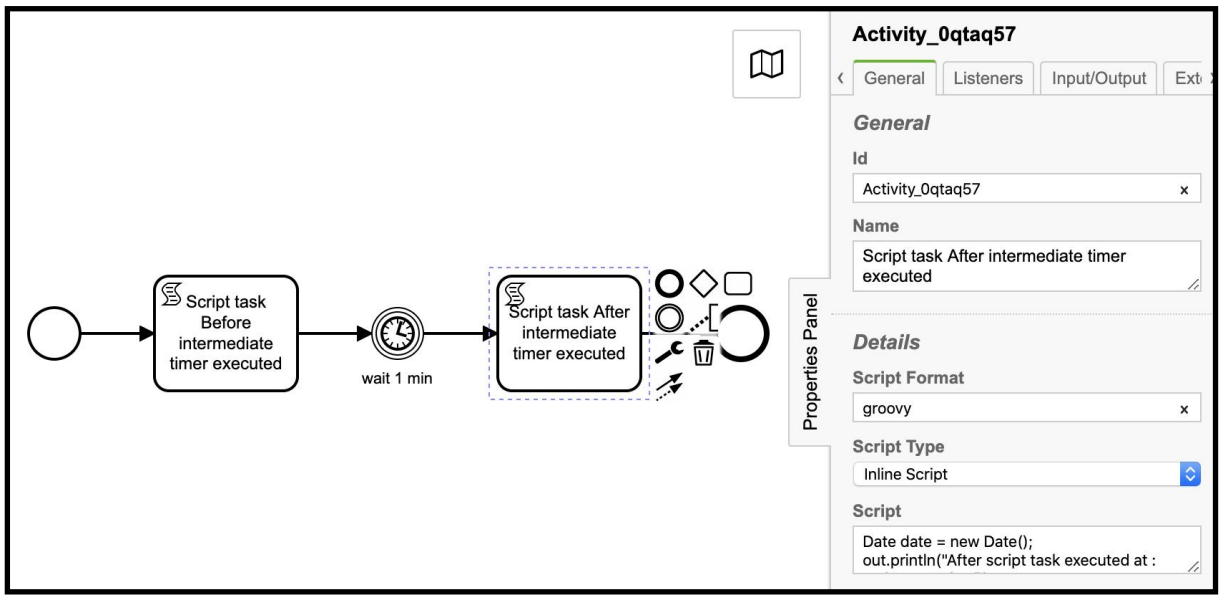
Id: Event_0s2nkr6

Name: wait 1 min

Timer Definition Type: Duration

Timer Definition: PT1M

2.3 Script task: Script task After intermediate timer executed



Id: Activity_0qtaq57
Name: Script task After intermediate timer executed
Script Format: groovy
Script Type: Inline Script
Script:

```
Date date = new Date();
out.println("After script task executed at : “+date.toString());
```

Deploy and start the process and observe console output

Before script task executed at : Mon Aug 31 12:25:17 PDT 2020
After script task executed at : Mon Aug 31 12:26:23 PDT 2020

3. Timer boundary event

A timer boundary event acts as a stopwatch and as an alarm clock. When an execution arrives in the activity to which the boundary event is attached, a timer is started. When the timer fires (e.g., after a specified interval), the activity is interrupted and the sequence flow going out of the timer boundary event are followed.

Interrupting Vs Non interrupting timer event

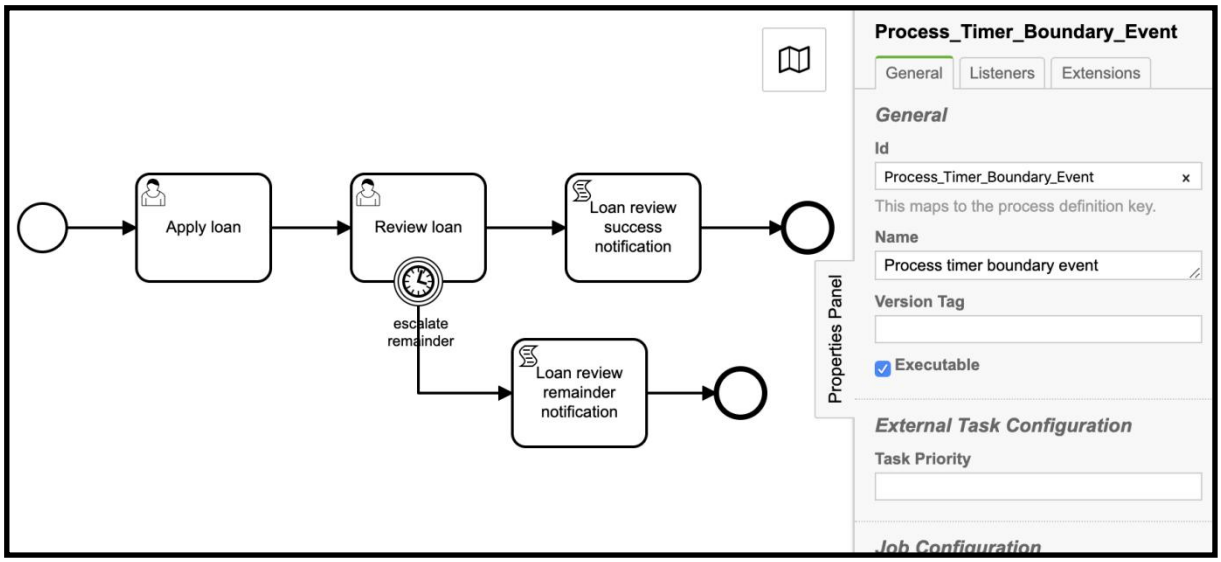
Cancel (Interrupting) timer event:

The interrupting event is the default. In the case of an interrupting Event, the activity which was being performed will immediately be canceled.

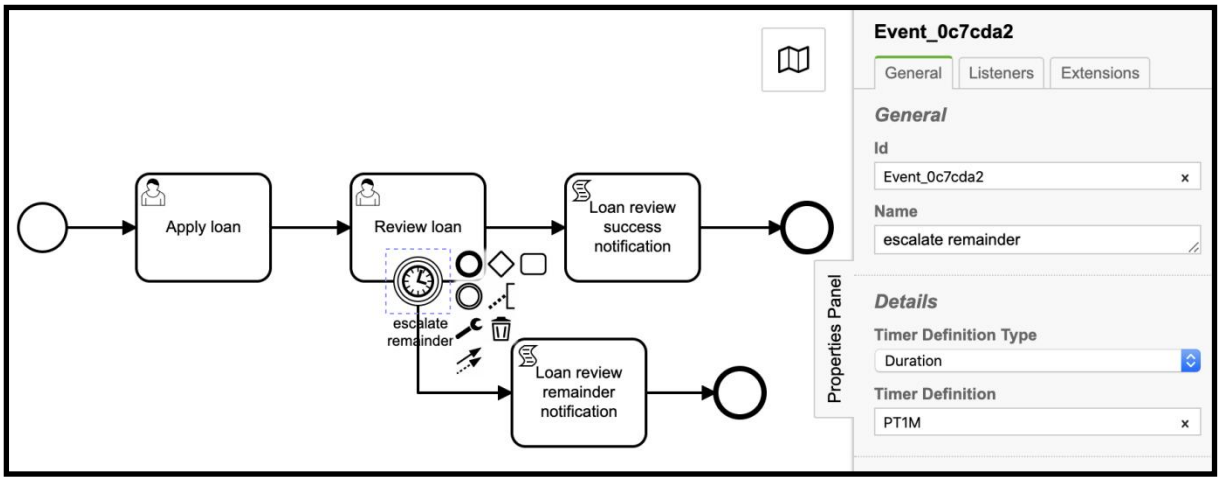
Non-interrupting timer event:

Non-Interrupting Event, the activity which was being performed will continue in parallel along with the new flow that was initiated by the boundary event. The current activity will NOT be cancelled (or interrupted).

Now let's model workflow using boundary timer event.

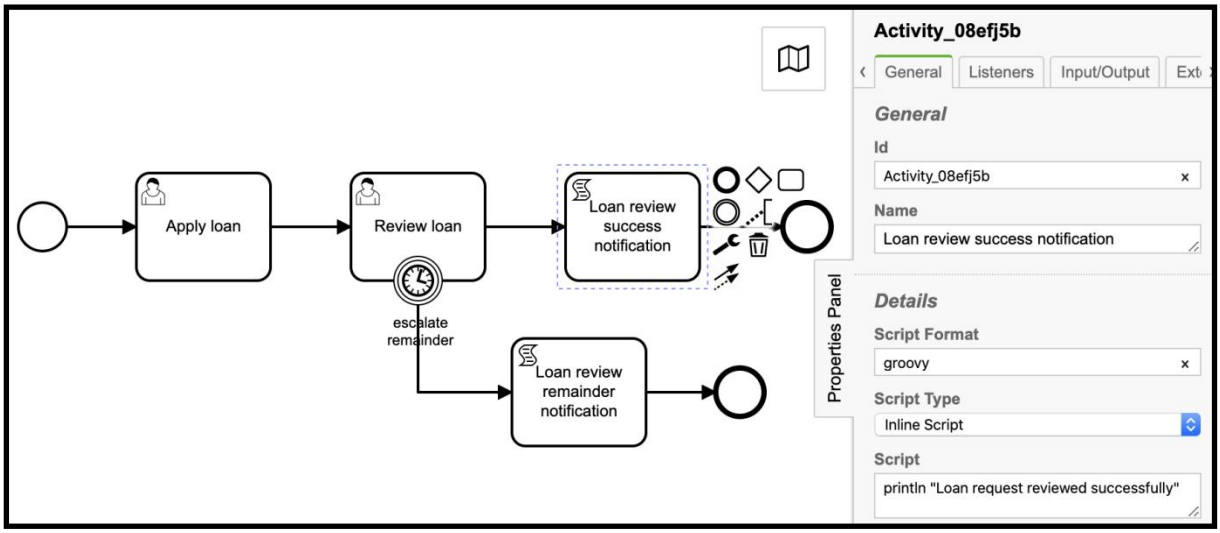


3.1 Boundary timer event (Interrupting)



Id: Event_0c7cda2
Name: escalate remainder
Timer Definition Type: Duration
Timer Definition: PT1M

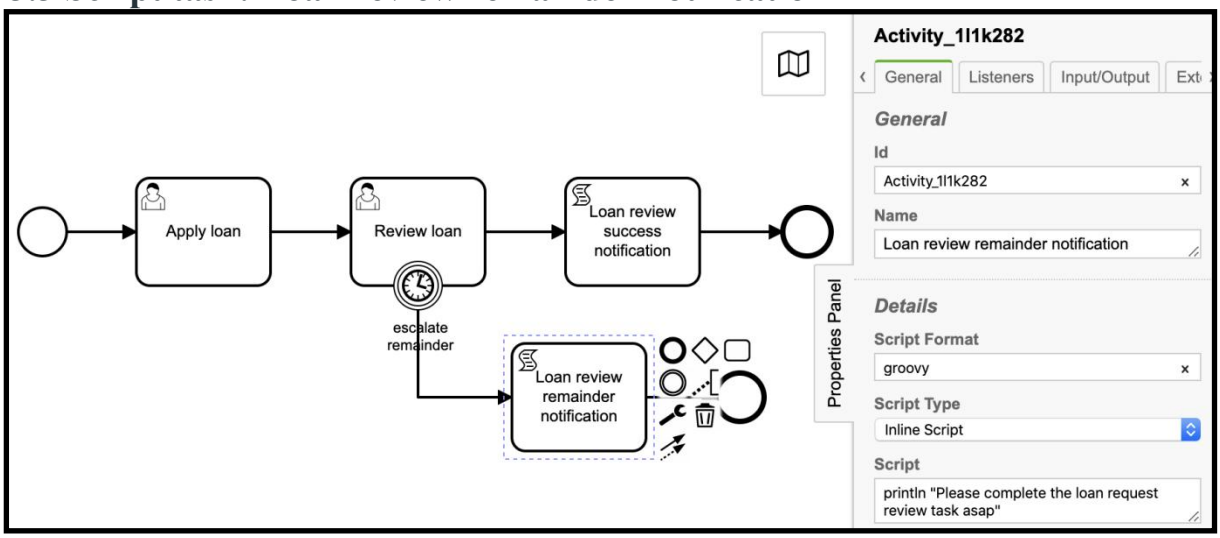
3.2 Script task: Loan review success notification



Id: Activity_08efj5b
Name: Loan review success notification
Script Format: groovy
Script Type: Inline Script
Script:

```
println "Loan request reviewed successfully”
```

3.3 Script task: Loan review remainder notification



Id: Activity_1l1k282
Name: Loan review remainder notification

Script Format: groovy
Script Type: Inline Script
Script:

```
println "Please complete the loan request review task asap"
```

Deploy and start the process.

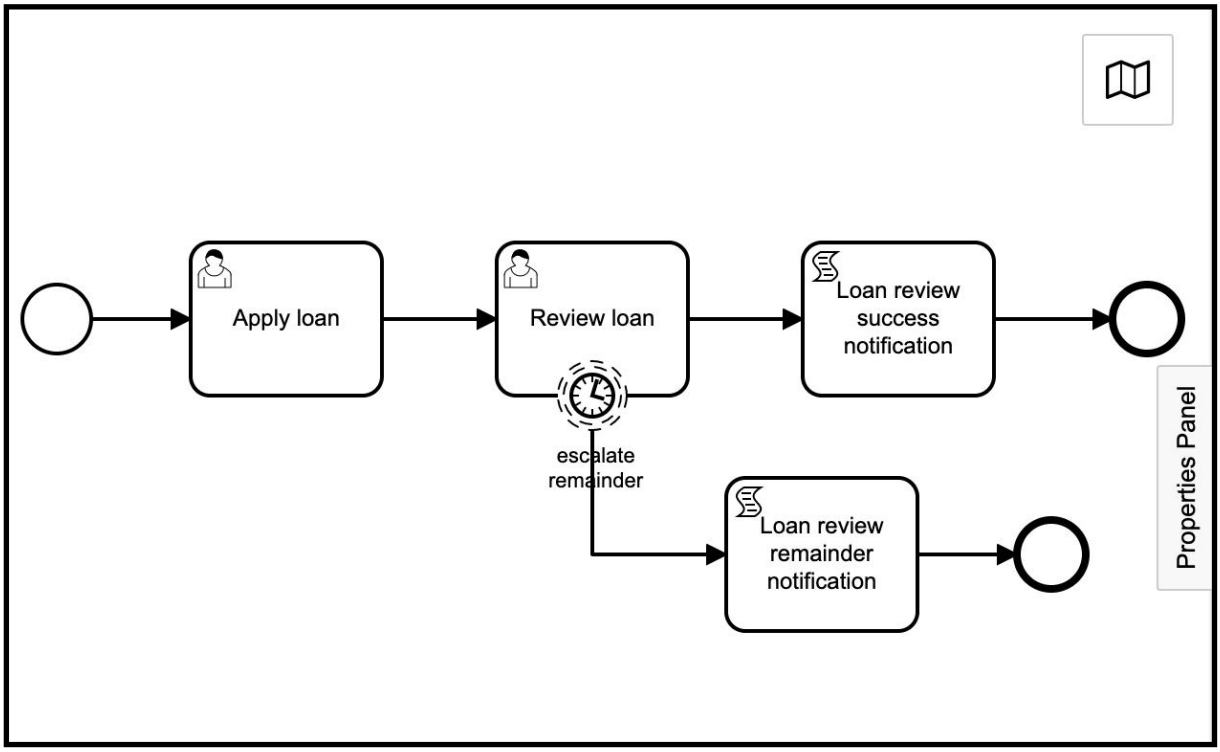
Complete user task Apply loan and then Review loan user task get created then wait for 1 min so that boundary timer will fire.

Observe the console output

Please complete the loan request review task asap

And then process instance is ended.

4 Boundary timer event (Non-Interrupting)



Deploy and start the process.

Complete Apply loan user task. User task Review loan will get created and wait till boundary timer event will get fired.

You will observe that user task Review loan will remain in user’s inbox even after remainder notification is sent.